Operating Systems and Parallel Computing

Lecture 15

Prof. Matteo Turilli

Where We Are

- Exercise 1 of Lecture 14: your job had 4 CPUs and an efficiency of 23%. One CPU worked and three waited.
- A Python program runs one instruction at a time. The operating system will not split it for us: the program has to start more **workers**.
- Today: two kinds of workers, threads and processes, and how to program them in Python, on the cluster.
- **Now**: open JupyterLab (`https://jupyter.iesci.tech:8085`), open a terminal (File > New > Terminal) and get today's notebook:

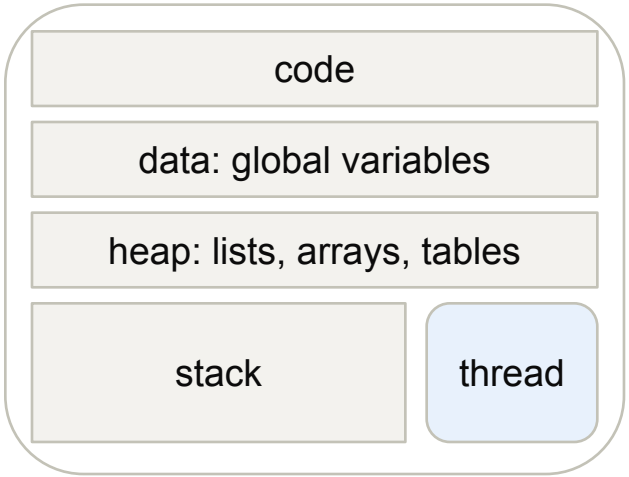
  ```
  curl -O https://mturilli.github.io/ospc-materials/notebooks/threads-processes.ipynb
  ```

- Double-click `threads-processes.ipynb` in the file list on the left. The top right corner of the notebook must say **OSPC (Python 3.12)**.

A Process Has Its Own Memory

- A **process** is a program in execution (Lecture 4). It has a number, the PID, and its own memory: code, data, heap and stack.
- The operating system keeps processes apart: one process cannot read or change the memory of another.
- `fork` creates a new process with a copy of the memory of the first one. From then on the two are separate.
- Every process starts with one thread: one line of execution, with its own position in the code and its own stack.

One process (PID 3025870)

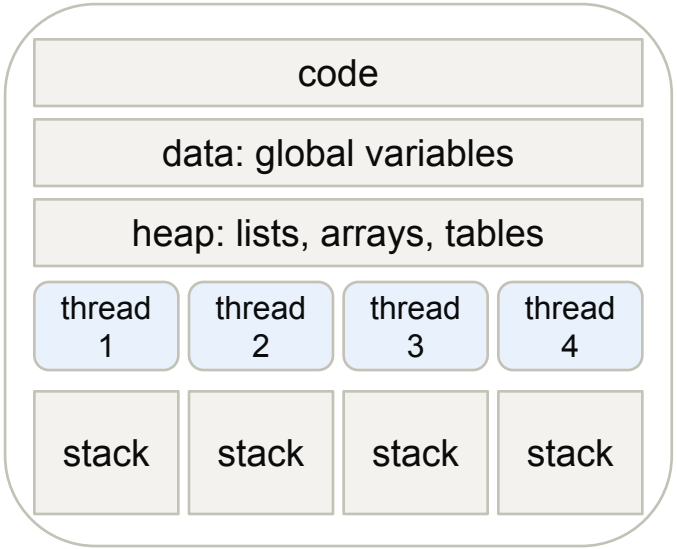


Another process cannot touch this memory.

A Thread Runs Inside a Process

- A **thread** is one line of execution inside a process (Lecture 5). A process can have many.
- Each thread has its own position in the code and its own stack.
- Stack: portion of the parent process memory page allocation, allocated to a thread. Logically isolated by the programmer, not by the MMU.
- All the threads of a process share its code, data and heap: when one thread changes a variable, it changes for all of them.
- The operating system schedules threads, so the threads of a process can run on different CPUs.

One process, four threads

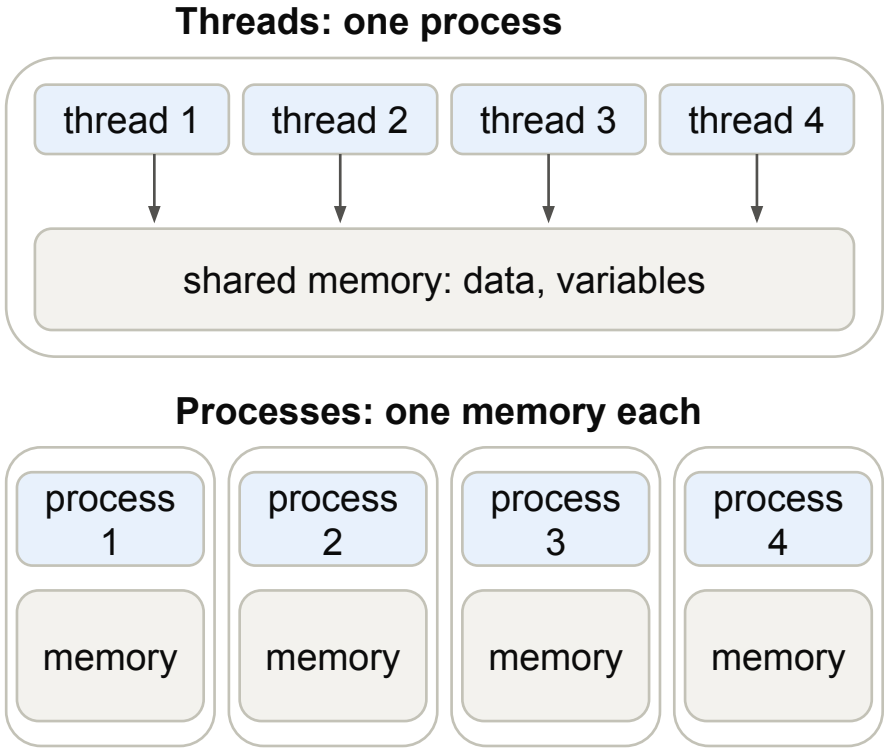


Code, data and heap are shared.
Each thread has its own stack.

Threads and Processes

	Threads	Processes
Memory	shared	one each
Data between workers	no copy	copied by the OS
A wrong write	can hit every thread	stays in one process
Start 4 and stop them	0.05 ms	3.3 ms
One small task and its result	0.006 ms	0.07 ms

Threads are light and share memory. Processes are heavier and share nothing unless we ask. Both are workers: Python programs them almost the same way.

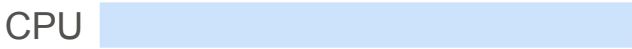


Data moves between processes as copies.

Computing or Waiting?

- Before we choose a worker, we ask what the program does most of the time.
- **Computing** (CPU-bound):
  - loops, calculations, model fits.
  - The program needs a CPU all the time.
- **Waiting** (I/O-bound):
  - web requests, databases, slow disks.
  - The program needs almost no CPU, it waits for somebody else (Lecture 11).
- The two types of workload need different workers. We start with waiting.

Computing: the CPU is busy all the time



Waiting: the CPU is free most of the time



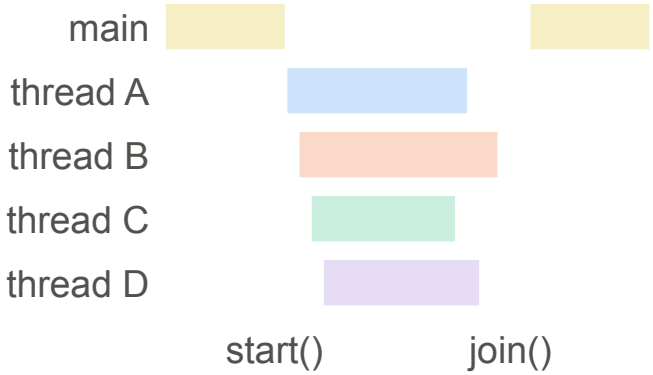
Python Threads: threading

```
import threading

def hello(name):
    print("hello from", name)

workers = [threading.Thread(target=hello,
args=(name,))
           for name in ["A", "B", "C", "D"]]

for t in workers:
    t.start()
for t in workers:
    t.join()
```



- `Thread(target=f, args=...)` creates a thread that will run `f`. Nothing runs yet.
- `start()` starts the thread and returns at once, so the main thread goes on.
- `join()` waits until that thread has finished.
- A thread cannot hand a `return` value back. It writes its result to shared memory.

Exercise 1: Threads

- Run **Part 1** of the notebook, up to “Stop here”. Click a cell and press Shift+Enter.
- You start four threads, you let them write to one shared list, and you time four downloads with and without threads.
- **Predict** at every cell marked Predict, before you run it.
- These cells run here, in your JupyterLab session: 2 CPUs, which are one core.
- Then answer the questions on the next slide.



Exercise 1: Questions

1. Did the four threads print the same process number? The same thread number?
2. How long did the four downloads take one after the other, and with 4 threads?
3. Your session has one core. How can four threads finish in 1 s?
4. What if `download` computed for 1 s instead of waiting? Would 4 threads still take 1 s?

Exercise 1: Answers

1. Did the four threads print the same process number? The same thread number?
2. How long did the four downloads take one after the other, and with 4 threads?
3. Your session has one core. How can four threads finish in 1 s?
4. What if `download` computed for 1 s instead of waiting? Would 4 threads still take 1 s?

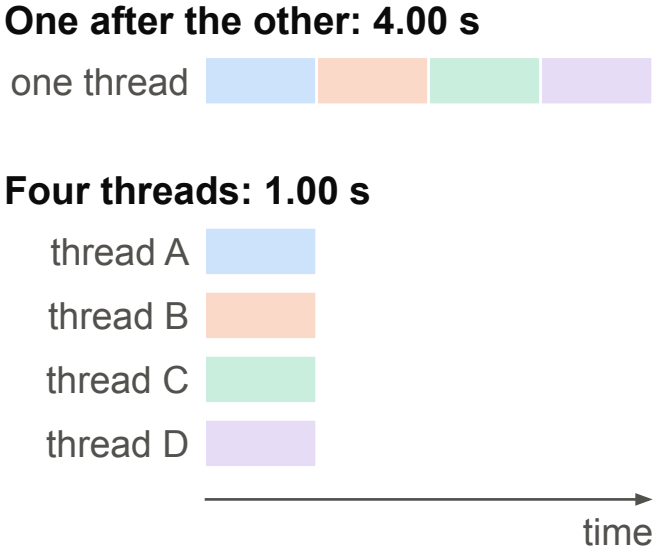
1. The same process number: they are all threads of the notebook's process. A different thread number each, and the main thread's number is the PID.
2. 4.00 s and 1.00 s.
3. A thread that waits needs no CPU. The core only has to start the waits, which takes almost no time.
4. No. Computing needs a CPU, and Python adds one more limit: the GIL.

Threads Can Wait Together

- All the threads see the same list, so each one writes its result there:

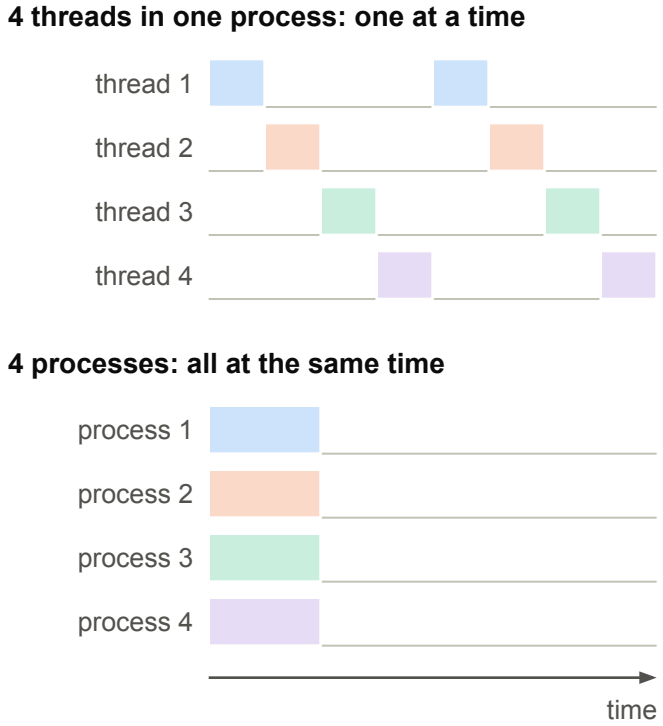
```
results = [None] * 4
def download(i):
    time.sleep(1)     # waiting for a web server
    results[i] = f"file {i}"
```

- A thread that waits uses no CPU, so four threads can wait at the same time, on one core.
- In your session, four (fake) “downloads” took 4.00s one after the other and 1.00s with four threads.



Python Threads Take Turns: the GIL

- The Python we use (CPython) has a **global interpreter lock**, the GIL. A thread must hold it to run Python code, and each process has one.
- So the threads of one process take turns, about 5 ms each: only one runs Python code at a time, however many cores there are.
- A thread lets go of the GIL while it waits: for a file, the network, or `time.sleep`. That is why four downloads took 1.00 s.
- Processes have one GIL each, so they run Python code at the same time, on different cores.
- Step through it: Threads in Python, step by step.



Each worker has the same work to do.

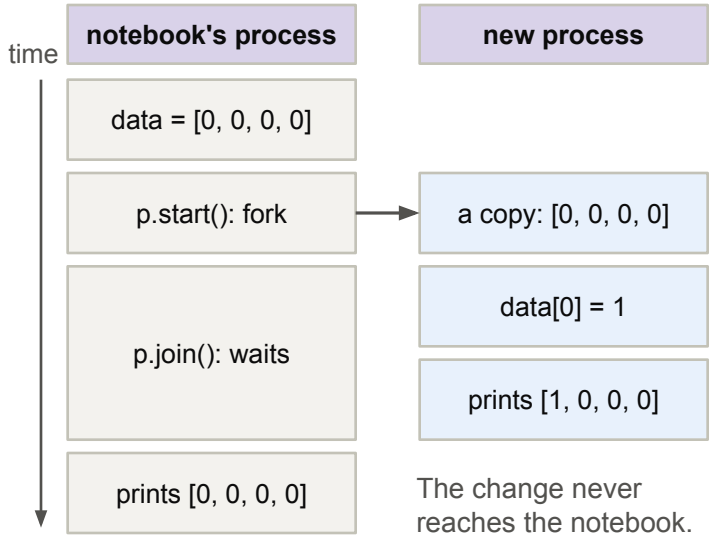
Python Processes: multiprocessing

```
import multiprocessing

data = [0, 0, 0, 0]

def change(i):
    data[i] = 1
    print("in the new process:", data)

p = multiprocessing.Process(target=change, args=(0,))
p.start()
p.join()
print("in the notebook:", data)
```



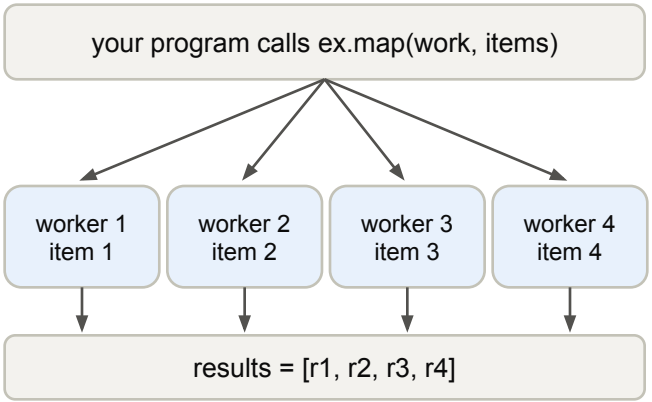
- `multiprocessing.Process` works like `threading.Thread`: `target`, `args`, `start()`, `join()`.
- On the cluster the new process starts with a copy of the memory, so the change stays in the new process.
- The notebook still prints `[0, 0, 0, 0]`.
- A result can only come back as a copy, through the operating system. A pool does that for us.

A Pool of Workers

```
from concurrent.futures import ProcessPoolExecutor

def work(n):
    total = 0
    for i in range(n):
        total += i
    return total

if __name__ == "__main__":
    items = [10_000_000] * 4
    with ProcessPoolExecutor(4) as ex:
        results = list(ex.map(work, items))
```



r1 is the result of item 1, and so on. Items and results travel between the processes as copies.

- `ProcessPoolExecutor(4)` starts 4 worker processes. They wait for tasks and stop at the end of `with`.
- `items` holds 4 numbers, so `ex.map` calls `work` 4 times, each time in a free worker, and returns the results in the order of the items.
- `ThreadPoolExecutor` works the same way, with threads.
- In a .py file, `if __name__ == "__main__":` keeps the pool in your program: the workers must not start pools of their own.

Exercise 2: Threads or Processes?

- Run **Part 2** of the notebook, up to “Stop here”.
- First a process by hand, then a pool here in your session, on one core.
- Then the notebook writes `compare.py` and submits it with `sbatch`: 4 CPUs, then 8. It runs four tasks one after the other, with 4 threads, with 2 processes and with 4 processes.

  ```
  !sbatch -p cpu -c 4 --mem=2G -t 2 -o compare-4cpus.out --wait
  --wrap "python compare.py"
  ```

- `--wait` keeps the cell at `[*]` until the job ends. The whole class shares the cluster: it can take up to half a minute.
- **Predict**: how many times faster than one after the other will each line be, with 4 CPUs? With 8?

Exercise 2: Questions

1. What did your job print with 4 CPUs and with 8? Are you on the same node as your neighbor?
2. Why were 4 threads no faster than one after the other, even with 8 CPUs?
3. Here in your session 4 processes gained nothing, and with 4 CPUs they were about 2 times faster. Why not 4 times?
4. Your script downloads 400 files from a web service, one after the other. Threads or processes? Why?

Exercise 2: Answers

1. What did your job print with 4 CPUs and with 8? Are you on the same node as your neighbor?
2. Why were 4 threads no faster than one after the other, even with 8 CPUs?
3. Here in your session 4 processes gained nothing, and with 4 CPUs they were about 2 times faster. Why not 4 times?
4. Your script downloads 400 files from a web service, one after the other. Threads or processes? Why?

1. On haskell, one after the other 0.73 s, 4 threads 0.71 s. With 4 CPUs: 2 processes 0.39 s, 4 processes 0.32 s. With 8 CPUs: 0.36 s and 0.18 s.
2. `work` runs Python code, so each thread needs the GIL. They took turns, one at a time.
3. Your session has 1 core and a job with 4 CPUs has 2. Workers can only use the cores that are there: next slide.
4. Threads: a thread that waits lets go of the GIL, so the waits overlap. Processes work too, but cost more to start.

Why 2 Times, Not 4?

Your job	Cores	One after the other	2 processes	4 processes	Times faster
JupyterLab, 2 CPUs	1	0.72 s	0.66 s	0.64 s	1.1
Batch job, 4 CPUs	2	0.73 s	0.39 s	0.32 s	2.3
Batch job, 8 CPUs	4	0.73 s	0.36 s	0.18 s	4

- Each core of our nodes runs two hardware threads, and Linux and Slurm count each one as a CPU. Slurm gives out whole cores: 4 CPUs are 2 cores.
- Two workers on one core share it, so each runs at about half speed. What counts for a pool is the number of cores.
- Step through it: A pool of processes, step by step.

Races: Two Workers, One Balance

- Four threads make 10,000 deposits each on one shared balance. The right total is 40,000.

```
def deposit(n):
    global balance
    for _ in range(n):
        current = balance      # read
        time.sleep(0)          # wait
        balance = current + 1  # write
```

- The wait stands for a database or a file. The thread lets go of the GIL, and the others run.
- When two workers read the same value, one update is lost (right): a **race** (Lecture 5).
- A **lock** lets one worker at a time through the three steps.

Two workers, one balance

Worker A	Worker B	balance
read 10		10
	read 10	10
write 11		11
	write 11	11

time

Two deposits, but the balance went from 10 to 11.

Exercise 3: Races and Locks

- Run **Part 3** of the notebook, up to “Stop here”.
- Four threads on a shared balance, with and without a lock.
- Four processes on a counter in shared memory, with and without a lock.
- Then four processes that share nothing.
- **Predict** the total before each cell, and run the first cell a few times.

Exercise 3: Questions

1. What balance did the threads reach without the lock, and with it? How long did each take?
2. Why was the balance about a quarter of 40,000?
3. There is no wait in `counter.value += 1`. Why did the four processes lose updates?
4. Which version was the fastest, and why?

Exercise 3: Answers

1. What balance did the threads reach without the lock, and with it? How long did each take?
2. Why was the balance about a quarter of 40,000?
3. There is no wait in `counter.value += 1`. Why did the four processes lose updates?
4. Which version was the fastest, and why?

1. About 10,000 in 0.46 s, and 40,000 in 2.03 s.
2. All four threads read the same value before any of them writes, so four deposits count as one.
3. It is three steps: read, add 1, write. The two CPUs of your session run at the same time, so two processes can read the same value.
4. Nothing shared: about 0.02 s, with no lock to wait for and no shared memory to reach.

Locks, or Nothing to Share

- A **lock** makes the read, the wait and the write one critical region (Lecture 5):

```
with lock:
    current = balance
    time.sleep(0)
    balance = current + 1
```

- The total is right, but the workers queue at the lock.
- Better: share nothing. Each worker counts on its own and returns its count, and the main process adds up the results.

```
total = sum(ex.map(count, [100_000] * 4))
```

- Split the work, let each worker compute its own part, and combine the results at the end.

In your session	Total	Time
Threads, no lock	about 10,000 of 40,000	0.46 s
Threads, lock	40,000	2.03 s
Processes, no lock	about 150,000 of 400,000	0.15 s
Processes, lock	400,000	0.56 s
Nothing shared	400,000	0.02 s

A 2-CPU session on haskell. This is too little work to gain from workers.

Parallelism Has Costs

What the pool does, in a 4-CPU job	Threads	Processes
Start 4 workers and stop them	0.05 ms	3.3 ms
Send one small task and get its result back	0.006 ms	0.07 ms
Hand an 80 MB NumPy array to a worker	no copy	117 ms
100,000 tiny tasks with `ex.map`; with `chunksize=1000`		4.65 s; 28 ms
16 tasks with 1 / 2 / 4 / 8 / 16 workers		1.41 / 0.74 / 0.65 / 0.65 / 0.66 s

- A task must take much longer than sending it: give each worker a few large tasks (`chunksize`), not many tiny ones.
- Send little data: every argument and every result is copied. Pass a file name and let the worker read the file.
- More workers than cores do not help (Lab 1 again): they take turns on the same cores.

How Many Workers? Ask Slurm

- Inside a 4-CPU job, `nproc` prints 4, but Python's `os.cpu_count()` prints 512: all the CPUs of the node.
- `ProcessPoolExecutor()` with no number starts one worker per CPU of the node. In our 4-CPU job it tried 512 workers, failed after 0.2 s with “Too many open files”, then hung until it was killed.
- Always give the number, and take it from the job:

```
n = len(os.sched_getaffinity(0))
with ProcessPoolExecutor(n) as ex:
```

- On our nodes two CPUs share one core: in a 4-CPU job, 4 processes ran 2.3 times faster than one after the other; with 8 CPUs, 4 times.

In a 4-CPU job on haskell	Prints
`nproc`	4
`echo $SLURM_CPUS_PER_TASK`	4
`os.cpu_count()`	512
`len(os.sched_getaffinity(0))`	4
`ProcessPoolExecutor()` workers	512

Which Workers for Which Work?

Your work	Use	Why
Python code: loops, your own functions	Processes	Each process has its own GIL
Waiting: downloads, web APIs, databases	Threads	A waiting thread lets go of the GIL
Many independent runs: parameters, folds, files	Processes, or one batch job per run	Nothing to share
NumPy matrix products, scikit-learn models	Their own threads and `n_jobs` first (Session 23)	They can already use several cores

- In every case: split the work into independent tasks, combine the results at the end, and size the pool to the CPUs of your job.
- Time one worker on one core first: it is the baseline of the speedup you report (Session 23).

Homework and Next Time

- **Notebook, Part 4**: run `speedup.py` with 8 CPUs and with 16, and keep your two tables for Session 23. Then try the many tiny tasks, and write down which step of your project is computing and which is waiting.
- **Project proposal**: due tonight, Wednesday 4 November, 23:59. Say which row of the parallelism map (Lecture 13) your workload is in.
- Read the Python documentation of `concurrent.futures` (docs.python.org/3/library/concurrent.futures.html), up to the ProcessPoolExecutor example.
- At home, step through the two pages of today and repeat what you did not finish. Your files stay in your home folder on the cluster.
- **Next time**: parallelism inside the tools (`n_jobs`, NumPy and its BLAS threads), then the scaling lab.